

Are LLM Web Search Engines Sustainable?

A Web-Measurement Study of Real-Time Fetching

Abdur-Rahman Ibrahim
Sayyid-Ali
abdurrahman.ali@lums.edu.pk
Lahore University of Management
Sciences (LUMS)
Lahore, Pakistan

Daanish Uddin Khan
25100004@lums.edu.pk
Lahore University of Management
Sciences (LUMS)
Lahore, Pakistan

Naveed Anwar Bhatti
naveed.bhatti@lums.edu.pk
Lahore University of Management
Sciences (LUMS)
Lahore, Pakistan

Abstract

Large language model (LLM) based answer engines offer real-time, conversational answers but raise concerns about scalability and web sustainability. Unlike traditional search that serves results from cached indices, these systems fetch pages anew for each query, creating redundant network traffic. We present the first measurement-driven study of commercial LLM answer engines, combining automated client-side tracing and controlled server-side audits. Analyzing ChatGPT and Claude across 1,000 queries, we find that both operate as meta-search layers heavily reliant on existing indices, fetching top-ranked pages with minimal caching. A human-equivalent cost model shows their per-query network footprint far exceeds that of human searchers, varying sharply by architecture. These results reveal the infrastructural burden of real-time fetching and motivate cooperative efficiency measures like shared caches, transparent retrieval standards, and publisher controls such as *llms.txt*, to make AI-augmented search more sustainable.

CCS Concepts

• **Information systems** → **Web search engines**; *Information retrieval*; *World Wide Web*; • **Networks** → **Network measurement**; • **Computing methodologies** → *Natural language processing*; • **Social and professional topics** → *Sustainability*.

Keywords

LLM-Based Search; Web Answer Engines; Real-Time Fetching; Web Measurement; Search Engine Sustainability; Network Footprint

ACM Reference Format:

Abdur-Rahman Ibrahim Sayyid-Ali, Daanish Uddin Khan, and Naveed Anwar Bhatti. 2026. Are LLM Web Search Engines Sustainable? A Web-Measurement Study of Real-Time Fetching. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792278>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18326112>.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792278>

1 Introduction

Large language model (LLM)-based *answer engines* promise search that feels less like a scavenger hunt and more like a crisp, conversational synthesis backed by citations. This paradigm also changes how the Web is touched. Instead of serving results from precomputed indices, many systems fetch content in real time, reading multiple pages per query to compose a response. This design can improve freshness and breadth, but it reintroduces costs that classic Web search spent decades taming through indexing, caching, and polite crawling [23]. The result is a measurable per-query retrieval footprint on the open Web.

This retrieval model has consequences at Internet scale. Traditional search amortizes data transfer across billions of users through centralized indices and cache hierarchies. In contrast, on-demand fetching can duplicate transfers for semantically similar questions. Telemetry reports already show automated traffic at historic levels in 2024, with bots at approximately 51% of global visits and “bad bots” at 37% [13, 14]. Observers attribute this growth in part to AI-driven crawlers and fetchers. Widespread LLM adoption magnifies these stakes. With hundreds of millions of daily queries and projections of more than 750 million LLM-integrated applications by end of 2025 [21, 25], per-query fetching could generate a massive volume of redundant page loads each year. This model shifts network and infrastructure costs directly onto source websites, particularly smaller publishers with limited capacity to absorb the load.

Despite this rapid proliferation, we lack an empirical understanding of how commercial LLM answer engines actually interact with the open Web. This knowledge gap has direct consequences for both infrastructure and the information economy. Without systematic data, we cannot quantify the strain on web servers, determine whether these answer engines simply piggyback on existing search indices (reinforcing the status quo) versus discovering new content, nor assess whether real-time fetching is sustainable or merely duplicates traditional crawling efforts at scale. The community also lacks evidence to guide potential efficiency interventions, for example, shared caching layers or opt-out standards like *llms.txt* for web publishers [11]. To address this gap, our work provides the first measurement-based account of LLM answer engine behavior, guided by the following research questions:

RQ1 To what degree do the URLs fetched by an LLM answer engine align with results in public search indices (for example, Google, Bing, and Brave), and what does this imply about independence versus reliance on existing indices?

- RQ2** How often do fetched sources appear as citations in the final answer, and what types of media (HTML pages, video, images, etc.) are retrieved (and cited) during the answer process?
- RQ3** How many distinct sources (domains/origins) are accessed per query, and how does this vary across different query intents (navigational vs. informational vs. transactional)?
- RQ4** To what extent is content retrieval cached or otherwise reused across similar queries to reduce downloads?

Approach and Contributions: We conduct a comparative, two-pronged black-box measurement of leading commercial LLM answer engines to answer these questions. First, we develop a user-server instrumentation pipeline that automates the engines’ web UI (using Puppeteer [9]) to record detailed client-side retrieval traces for each query. Second, we perform a server-side experiment on a controlled test website to audit the engine’s traversal, content extraction, and caching behavior under known conditions. Using 1,000 queries from the ORCAS-I benchmark [1] (spanning multiple intent categories), we analyze the engines’ accessed versus cited URLs, their overlap with standard search results, and any signs of result re-use. At a high level, our measurements reveal that LLM answer engines touch far more pages per query than a human searcher would, heavily (but not entirely) leverage the same results found in public search indices, and perform little caching, resulting in a lot of repeated fetching that leaves considerable room for efficiency gains. Building on these findings, we make the following contributions:

- **A reproducible pipeline** for UI that separates the search phase from answer generation and enumerates accessed and cited URLs at answer time (Section 4.1).
- **An intent-aware analysis** across 1,000 ORCAS-I queries that quantifies pages touched per answer, per-query network cost, and alignment of fetched URLs with public search results (Section 4.2)
- **A controlled audit** that surfaces practical limits in dynamic content handling, media processing, and caching, with implications for both coverage and bandwidth (Section 5.2).
- **Actionable guidance** based on concrete metrics such as pages touched per query and search strings per query, and design implications including caching and standards like `llms.txt` to reduce redundant retrieval (Section 5.3 and 7).

Taken together, these contributions provide the first data-driven insight into the web-scale impacts of LLM-based search. Our study proposes a blueprint to design more efficient LLM answer engines that preserve the benefits of up-to-date, conversational answers while mitigating strain on the Web’s infrastructure and ecosystem.

2 Related Work

Research on LLM-based answer engines spans several interrelated areas.

LLM-augmented search architectures: A line of systems integrates LLMs with live web search and browsing to improve answer quality and freshness [10, 18, 28, 32]. Early prototypes such as WebGPT augmented an LLM with browsing to produce source-cited answers [22]; subsequent systems (e.g., WebGLM and AutoWebGLM) enable multi-step navigation and reasoning across

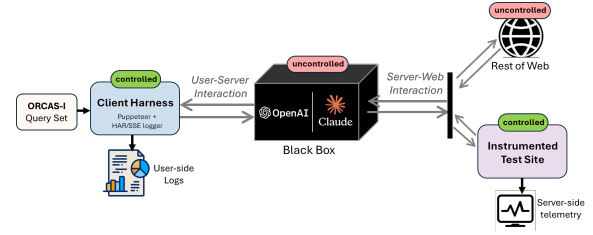


Figure 1: Methodology

pages [4, 17, 20, 39], while retrieval-augmented methods like FRESH-LLMs fetch documents at query time to boost accuracy [36]. These efforts demonstrate capability but implicitly incur significant retrieval overhead by loading multiple pages per query. Our work is the first to *measure* this overhead at scale, quantifying the network and infrastructure costs of on-demand LLM search. We therefore shift the question from “*Can LLMs browse for better answers?*” to “*What are the web-wide implications when they do?*”

Crawler governance and web protocols for AI: The rise of LLM-powered agents has prompted proposals to regulate their web access beyond `robots.txt`. New standards such as `llms.txt` and the more expressive `ai.txt` aim to let publishers opt out of or guide AI crawlers [11, 40]. Implementation remains challenging: many scrapers fail to honor crawler directives [3], and studies of site owners report low awareness and limited agency regarding AI traffic [27]. Our measurements reinforce the need for cooperative, transparent protocols: by documenting redundant page requests generated by an answer engine, we provide quantitative motivation for governance to prevent undue strain on web infrastructure.

User-facing evaluation of answer engines: Parallel work evaluates these systems for correctness, credibility, and UX [7, 12, 19, 29]. Audits of early LLM-based search report hallucinated or misattributed citations and overconfident summaries [30], with follow-up design recommendations [31]. Domain studies find that, for medical questions, unguided LLMs often err unless grounded in trusted sources [8], and news-query analyses surface narrow, skewed citation patterns [38]. Large-scale usability work shows generative search can speed task completion but may reduce engagement with original content [16]. Whereas these studies center answer quality and interface design, we examine the hidden *operational* layer: not whether a response is correct, but *what it costs the web* for the LLM to assemble it.

Prior research has (i) engineered LLM+web prototypes, (ii) assessed answer quality and UX, and (iii) proposed governance standards. None, however, has empirically quantified the *network-level or web-scale impact* of real-time fetching by LLM answer engines. We fill this gap with the first at-scale measurements of bandwidth and server-load implications of this paradigm, reframing the sustainability of LLM-based web search as a first-class concern alongside accuracy and credibility.

3 Measurement Framework

To empirically quantify the web-scale retrieval behaviors of LLM answer engines, we designed a comprehensive, black-box measurement framework. Since the internal architectures of these systems

are proprietary, our methodology focuses on analyzing their observable interactions with the open web. Our framework employs two complementary experimental approaches: ① a large-scale, user-side measurement to capture retrieval patterns across a diverse set of real-world queries, and ② a controlled, server-side audit to probe specific system capabilities and limitations.

Our study focuses on two leading answer engines: ChatGPT (OpenAI) and Claude (Anthropic). These platforms represent the state-of-the-art and are widely adopted [26, 34]. We initially planned to include Grok [37] and Qwen [24], but they were excluded from our analysis as their public interfaces do not expose retrieval-related information (e.g., fetched URLs or search strings) as transparently as ChatGPT and Claude, precluding a direct and fair comparison of their retrieval footprint. The dual-methodology approach, illustrated in Figure 1, allows us to triangulate findings, providing a holistic view of how these engines touch the web.

3.1 Web Search Query Set

The foundation of our evaluation is the ORCAS-I dataset [1], an annotated extension of the ORCAS Bing search logs [5]. ORCAS-I provides a large collection of real user queries paired with intent labels, capturing the breadth and diversity of everyday search behavior without including personal identifiers. In total, it supplies 1,000 *gold set* queries, each categorized into a top-level intent (navigational, transactional, or informational) and, for informational queries, a finer subclass (factual, instrumental, or abstain)[1, 2]. Its construction applies stringent filtering to preserve user privacy, retain representativeness, and ensure the inclusion of only URLs present in the TREC Deep Learning corpus [6].

Figure 2 shows the distribution of these queries across intents. This rich, labeled query set is ideal for our purposes: it reflects authentic information needs and allows stratified analysis by intent type. By using ORCAS-I, we can examine whether different query intents lead to different retrieval strategies (e.g., do informational questions trigger broader web browsing than simple navigational look-ups?). In short, ORCAS-I’s scale and structure make it a highly suitable benchmark for evaluating LLM-based search engines under realistic conditions.

3.2 User-Server: Methodology

Our first experimental approach observes the LLMs’ web retrieval behavior from the user’s side. We aim to capture how ChatGPT (models: o4-mini, gpt-5, gpt-5-thinking) and Claude (models: Sonnet-4, Opus-4.1) perform live web searches on real queries, specifically, how they browse and what they retrieve for each question. By instrumenting the official browser-based interfaces, we ensure the evaluation runs under authentic conditions that mirror a normal user’s experience. This setup allows us to address key research questions about the structure of the LLMs’ search workflow [RQ1] and the breadth of content they fetch per query [RQ3]. We use the ORCAS-I 1,000-query gold set introduced in Section 3.1 for all user-side experiments (see Fig. 2).

3.2.1 Automated GUI Execution. We executed each query through the LLM’s web interface using an automated Chrome browser (via Puppeteer [9]) to precisely reproduce a typical user session. The

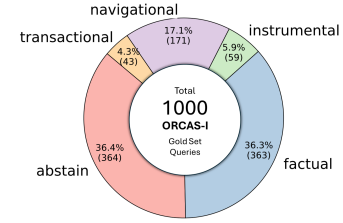


Figure 2: Distribution of 1,000 ORCAS-I gold-set queries by primary intent and informational subcategories.

automation script logs into the service, enables the model’s web-browsing mode, enters the query, and then waits for the answer to fully render, including any cited sources. We fixed the browser’s environment (viewport size, user-agent, locale) for consistency. By driving the actual GUI, our measurement captures the true sequence of network events and page loads that the user would trigger. This approach ensures we observe the native browsing behavior of each model, as opposed to relying on any unofficial API. In short, the user-side instrumentation lets us watch the models “surf the web” in real time under realistic conditions, yielding faithful data on what URLs they fetch and how long and how much data those fetches entail.

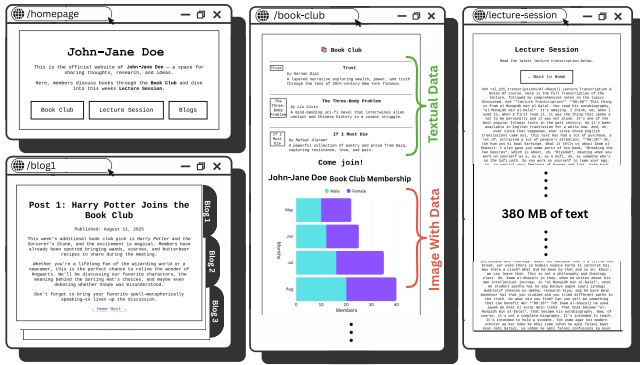
Importantly, we standardized the prompt format between models to ensure both ChatGPT and Claude consistently perform a web search for every query. ChatGPT provides an explicit “Web Search” toggle; when enabled, it attempts a web search for each user prompt. Claude’s interface, on the other hand, decides contextually whether to search. To force Claude to always browse, we prefixed every query with: “Search the web for: <query>”. This simple instruction reliably triggers Claude’s web mode for each prompt. It aligns Claude’s behavior with ChatGPT’s and guarantees that all 1,000 queries result in an attempted web search. We log both the original query and the full prompted string for completeness.

3.2.2 Network Capture and Data Logging. During each query session, we recorded the complete network trace to analyze the LLM’s retrieval steps. In practice, we captured a HAR file (HTTP Archive) [35] for the browser session as well as the streaming events from the model’s answer. Both ChatGPT and Claude expose a stream of Server-Sent Events (SSE) that delineate different phases of the answer. For ChatGPT, we tapped the SSE on the `/conversation` endpoint [33]; for Claude, the `/completion` SSE. These streams allowed us to partition each session into a search phase (when the LLM is actively retrieving content) and an answer phase (when it is composing the final answer). From the SSE and HAR data, we extract three critical pieces of information per query:

- **Accessed URLs:** the list of all distinct web URLs fetched by the LLM during its search phase (i.e., pages it opened while gathering information).
- **Cited URLs:** the subset of URLs that the LLM actually showed to the user as citations or references in its final answer.
- **Search Query Variations:** any search keywords or queries that the LLM reformulated internally and issued to the search engine. (Both ChatGPT and Claude often transform the user’s question into one or more search-engine queries; these appear in the streamed data, except in cases noted below.)

Table 1: Overview of Server-Web Tests

Test Name	Goal	Representative Query	Expected Output
Basic Data Test	Assess whether answer engine can retrieve simple structured data from a single page.	"What books are currently in the {website_name} bookclub?"	A list of five books from the book-club page.
Multi-Page Data Test	Evaluate ability to aggregate information across multiple pages.	"Find out all the books that are currently in the {website_name} bookclub."	Same five books as above, plus one additional from the blog pages.
Media Data Test (With contextual metadata of image)	Investigate whether the answer engine can interpret structured information embedded in an image.	"How many members does the {website_name} book club have?"	Report of membership count from an image.
Media Data Test (Without contextual metadata of image)	Examine handling of images lacking textual descriptors.	"What are the main sources of income for the {website_name} book club?"	Report the percentages and currency amounts of the different contributors.
Media Data Test (Audio)	Assess ability to locate and interpret non-image media such as audio.	"Are there any quotes by the founder of the namaloom frad book club on their website?"	Quote transcription from audio snippet.
Data Size Test	Explore how much content (in size) the answer engine processes when extracting a info from a page.	"What is this week's {website_name} lecture session about? ... favorite food place?"	Specific detail (restaurant preference) extracted from lecture session page.
Caching Test	Investigate whether the answer engine reuses cached web data across sessions.	No fixed query (behavioral observation).	Evidence of cached page retrieval within a short interval.

**Figure 3: Instrumented site for server-side tests: /book-club (mixed-media), blog (multi-step navigation), /lecture-session (large-scale processing limits)**

Using custom parsing scripts, we processed each session’s HAR+SSE into a structured JSON record containing the above fields along with metadata (timestamp, the model version/build, etc.). We deduplicated URL lists (while preserving order of access) so that if a model visited the same link multiple times during a session, it is counted once. This log of accessed and cited pages, along with query reformulations, underpins our analysis of LLM retrieval behavior.

3.2.3 Search-Engine Baseline Comparison. To contextualize the LLMs’ retrieval choices, we compare them against conventional search engine results. For each ORCAS-I query (and for each additional search-string variation the LLM disclosed), we issued the same text to three search engines: Google, Bing, and Brave. We scraped the top- K results from each engine’s SERP (with $K = 500$ in our experiments) and recorded the list of result URLs and their rank positions. We then evaluated the overlap between what the LLM fetched and what a search engine would have returned:

- For each URL that the LLM accessed, we check if that URL appears in the top- K results from Google, Bing, or Brave. If so, we note it as a hit and record the rank.
- We aggregate these hits to compute an index coverage rate: the percentage of the LLM’s accessed URLs that were present in a each search engine’s top results.
- We also examine the distribution of rank positions for those overlaps to see whether the LLM tends to retrieve pages that

rank near the top of search results, or whether it often brings in lower-ranked (more obscure) pages.

Comparing LLM retrievals with search engine indices allows us to test our central hypothesis. We analyze two factors: first, the simple overlap of URLs, and second, the distribution of their SERP ranks. The rank position analysis is critical: if LLMs were independently discovering content, we might expect significant URL overlap among popular sources, but a random distribution of ranks. Conversely, a retrieval pattern heavily skewed toward the top-ranked results provides strong, direct evidence that LLMs are programmatically using the SERP from a conventional search engine as their primary source list.

3.3 Server-Side Methodology

Our second experimental approach examines LLM web browsing from the web server’s perspective. In essence, we set up an instrumented test website and observed how the LLM interacts with it under various scripted scenarios. This controlled setting allows us to answer questions about the LLM’s capabilities that are difficult to isolate in the wild (such as how it handles multi-step navigation, non-text media, or caching)[RQ2][RQ4].

We built a small social community website (designed as a “book club” site with multiple sections) and deployed it on an local server. To allow for organic discovery and indexing by search engines, we waited 1.5 weeks after deployment before commencing our tests. The site’s pages included a homepage with club information, a series of blog posts, a page listing upcoming lecture sessions, and others, each page populated with specific content to test a particular aspect of LLM retrieval. For example, the “book club” page contained a mix of text and images to test data extraction from multi-modal content; the blog pages required clicking through pagination (to test navigation depth); the lecture page was intentionally very large (to test how the model handles large documents); and so on. Figure 3 provides an overview of this test website and the role of each page. All server requests were handled by Nginx with detailed logging, so we could record every access by the LLM, including timing and request headers.

Using this setup, we devised a suite of targeted tests. Each test corresponded to a specific research question about LLM behavior and came with a representative query that we posed to the LLM (ChatGPT, in this case) through its interface. We then observed how

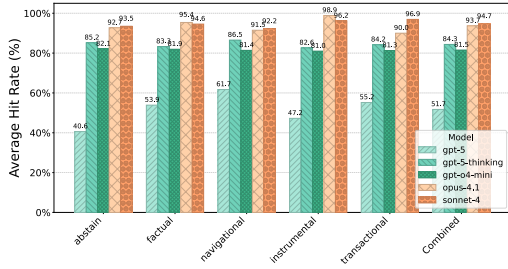


Figure 4: Average hit rate per query category for models: gpt-5, gpt-5-thinking, gpt-o4-mini, opus-4.1, and sonnet-4.

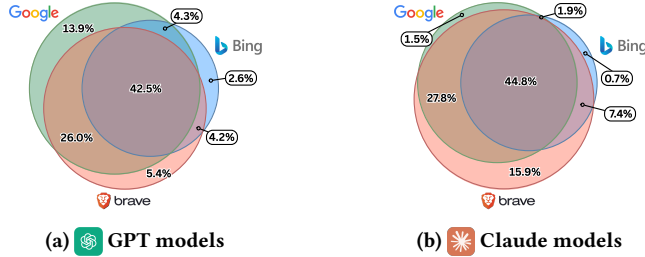


Figure 5: Aggregated URL overlap by provider (GPT left, Claude right). Each Venn diagram shows relative percentages of URL overlaps across Google, Bing, and Brave.

the LLM browsed our site and whether it succeeded in returning the expected result. We ran each test scenario in isolation and monitored every request the LLM made to our server, along with the content of its final answer. Table 1 summarizes these scenarios, their goals, example queries, and the expected correct behavior.

One of the most critical scenarios was the *Caching Test*, designed to probe for retrieval persistence and directly address [RQ4]. The methodology involved issuing a query that required access to a specific page on our test site. We then repeated the exact same query from the same IP address at timed intervals, first within a few minutes, and by analyzing the Nginx server logs for new page requests, we could empirically determine if, and for how long, the answer engine was reusing cached data rather than re-fetching the content from our server.

This controlled “server-side audit” allowed us to spot specific limitations (for instance, an inability to extract text from images, or repeated fetching of the same page in quick succession) under reproducible conditions. By combining these findings with the large-scale measurements, we can distinguish between systemic, web-wide issues and quirks or inefficiencies that manifest even on a single site.

4 Results: User-Server Interaction

Our user-side measurement study reveals that LLM answer engines are not independent web crawlers but rather meta-search systems whose retrieval strategies have profound and divergent impacts on the web. We structure our findings around two core themes: their reliance on existing search indices and the efficiency of their retrieval workflows.

Table 2: Average hit rate by model and search engine.

	Models				
	gpt-o4-mini	gpt-5-thinking	gpt-5	sonnet-4	opus-4.1
Bing	abstain	46.4%	51.3%	22.0%	50.8%
	factual	49.7%	55.0%	32.2%	56.2%
	navigational	41.0%	56.0%	40.8%	49.0%
	instrumental	40.8%	34.3%	23.3%	43.5%
	transactional	44.5%	40.0%	32.7%	46.2%
	Average	44.5%	47.3%	30.2%	49.1%
Google	abstain	74.3%	72.2%	41.1%	73.8%
	factual	75.8%	71.6%	53.6%	75.6%
	navigational	74.7%	78.8%	66.0%	67.9%
	instrumental	69.0%	74.6%	57.0%	72.3%
	transactional	72.5%	73.7%	60.0%	62.3%
	Average	73.3%	74.2%	55.5%	70.4%
brave	abstain	66.2%	69.0%	38.9%	89.9%
	factual	68.1%	66.3%	49.6%	90.1%
	navigational	60.7%	69.1%	59.9%	88.3%
	instrumental	67.0%	54.7%	45.5%	95.0%
	transactional	65.8%	63.6%	51.2%	96.2%
	Average	65.6%	64.6%	49.0%	91.9%

4.1 Reliance on Existing Search Indices [RQ1]

Our analysis provides strong evidence that LLM engines operate not as independent crawlers, but as meta-search systems that heavily leverage the ranked results of existing search engines. The first and most direct indicator is the high percentage of URL overlap, or “hit rate,” between LLM-fetched URLs and the top 500 results of conventional search engines. As shown in Figure 4, the combined hit rate (aggregated across Google, Bing, and Brave) is remarkably high for all models. Claude models (opus-4.1, sonnet-4) exhibit the strongest alignment, with combined hit rates exceeding 88%, while GPT models¹ (gpt-o4-mini, gpt-5-thinking) also show high alignment, averaging over 80%.

However, this aggregated view masks the distinct provider dependencies that drive these high scores. Table 2 provides a detailed, per-engine breakdown of these hit rates, revealing a crucial difference in strategy. For Claude models, the exceptionally high alignment is almost entirely attributable to a single provider: Brave. For instance, the hit rate for Claude models with Brave alone is over 90%, nearly matching their total combined hit rate. This indicates a deep and primary reliance on a single index. In contrast, while GPT models also have high combined coverage, their hit rates are more distributed across different search engines, and their alignment with any single engine is significantly lower than their combined total. This underscores that while both are meta-search systems, their methods for leveraging source indices are fundamentally different.

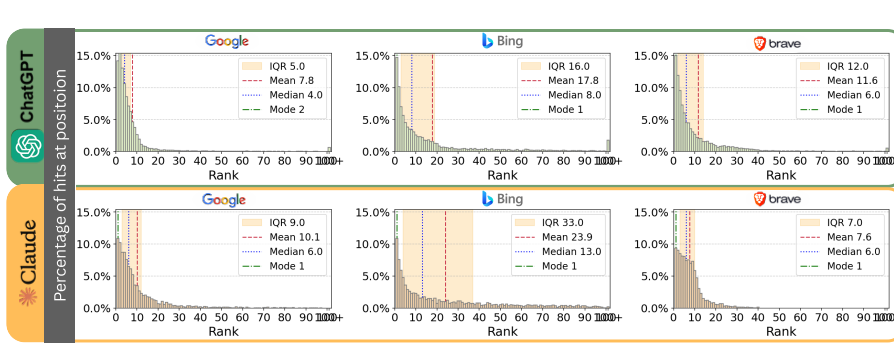
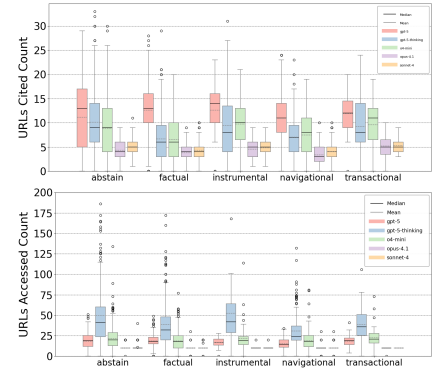
Figure 5 provides a more granular view of this reliance, revealing distinct retrieval strategies. The Venn diagrams show that while both providers leverage a common core of popular results (42–45% of URLs appear in all three search engines), their strategies for sourcing unique content diverge. GPT models source unique URLs primarily from Google (13.9%), whereas Claude models rely heavily on Brave (15.9%), indicating provider-specific preferences for their underlying search tools.

However, the most conclusive evidence of this dependency comes from our rank position analysis (Figure 6). This analysis reveals that the fetched URLs are present in search results, and are overwhelmingly concentrated at the very top of the SERPs. For GPT models querying Google, the mean rank is 7.8 (median 4.0), while

¹Unlike the other models, gpt-5 did not disclose its internal reformulated search strings in the interface, meaning alignment can only be measured against the single user prompt. It may still generate such queries internally, but this limitation narrows the comparability of its hit-rate values.

Table 3: Average number of URLs cited, URLs accessed, and search string counts by category and model.

Category	GPT-5			GPT-5-Thinking			GPT-o4-Mini			Opus-4.1			Sonnet-4		
	URLs Accessed	URLs Cited	Search Count	URLs Accessed	URLs Cited	Search Count	URLs Accessed	URLs Cited	Search Count	URLs Accessed	URLs Cited	Search Count	URLs Accessed	URLs Cited	Search Count
Abstain	17.93	11.18	1.00	49.88	10.11	10.06	21.79	8.80	4.56	10.14	4.32	1.01	10.28	5.05	1.03
Factual	18.78	12.61	1.00	38.29	6.69	7.77	18.57	6.55	3.72	9.86	3.80	0.99	10.10	4.32	1.01
Navigational	15.46	10.95	1.00	30.82	7.05	6.64	19.36	7.47	3.89	9.77	3.38	0.98	10.12	3.99	1.02
Instrumental	17.08	12.61	0.98	52.42	9.46	10.24	21.98	9.76	5.00	10.17	4.54	1.02	10.17	4.85	1.02
Transactional	18.40	11.93	1.09	38.70	9.28	8.65	22.58	9.60	4.26	9.77	5.23	0.98	10.00	5.33	1.00

**Figure 6: Distribution of hit rate by SERP rank (combining distribution of all gpt and claude models into one respectively), revealing a heavy concentration on top-ranked results. The strong right-skew demonstrates that engines primarily amplify content from the first page. The tightest distributions for GPT models on Google and Claude models on Brave highlight different underlying index preferences.****Figure 7: Distribution of URLs accessed (top) and cited (bottom) per category (gpt-o4-mini vs. gpt-5-thinking vs. gpt-5 vs. opus 4.1 vs. sonnet 4).**

for Claude on Brave, 75% of URLs originate from the top 10 results. This consistent, top-heavy retrieval pattern is improbable to occur by chance through independent discovery. Instead, it strongly indicates a systematic process of querying conventional search engines and preferentially fetching highest-ranked results, a behavior that demonstrates engines primarily amplify already popular content.

Furthermore, this deep reliance on search indices is not situational but is a universal strategy applied across all query intents. As shown in Figure 4, the hit rates for all models remain consistently high regardless of whether the query is navigational, factual, or instrumental. Claude models, in particular, show an almost flat alignment above 88%, indicating their retrieval strategy does not adapt to query type. This intent-agnostic behavior, combined with the rank position analysis, provides conclusive evidence that these engines operate as meta-search layers that systematically amplify the web's already most visible content.

4.2 Retrieval Volume and Efficiency [RQ3]

While both engine families rely on indices, their retrieval strategies diverge dramatically, directly impacting their web footprint and answering RQ3. The consistent behavior within provider families suggests the use of a shared retrieval tool.² Figure 7 and Table 3 illustrate a clear architectural contrast:

- **Exploratory Retrieval (GPT):** Models like gpt-5-thinking exhibit a "heavy-tailed" retrieval pattern, a characteristic clearly visible in the wide distributions and frequent outliers for accessed URLs shown in Figure 7 (top). Our intent-based analysis reveals

this behavior is not random but is driven by query complexity. As detailed in Table 3, when handling open-ended "instrumental" or "abstain" queries, gpt-5-thinking's workload dramatically increases, issuing over 10 search strings to access an average of over 50 pages. This is a significant expansion compared to the ~7 strings and ~38 pages used for simpler factual queries, explaining the high-cost retrieval patterns and lower overall efficiency.

- **Constrained Retrieval (Claude):** In stark contrast, Claude models are architecturally constrained and intent-agnostic in their volume, resulting in the tightly bounded and predictable retrieval footprint shown in Figure 7. As the data in Table 3 confirms, they retrieve a fixed set of ~10 URLs driven by a single search string for every query category. This rigid, fixed-cost approach ensures high efficiency (consistently citing 4–5 of the 10 pages it accesses) but sacrifices the ability to adapt its exploratory breadth to the user's information need.

This difference in retrieval volume leads to a crucial disparity in efficiency, which we measure with the *access-to-citation ratio*: the number of pages an engine fetches versus the number it ultimately presents to the user. The average values in Table 3 quantify this gap clearly. GPT-5-thinking shows the lowest efficiency, accessing 42.9 URLs on average to produce just 9.2 citations. The more compact gpt-o4-mini is better balanced, accessing 23.3 URLs to cite 9.0. By comparison, Claude's structurally limited model is inherently efficient, citing 4–5 of the 10 pages it accesses.

4.3 Takeaway

Our results paint a clear picture: LLM answer engines operate as meta-search layers that repackaging the web's most visible content

²User-side logs confirm this, indicating that all tested GPT models utilize the same web. run() tool for retrieval operations.

using one of two distinct strategies: GPT’s broad, inefficient, multi-query exploration versus Claude’s narrow, efficient, single-pass retrieval. This reveals a fundamental trade-off between the potential for comprehensive synthesis and the infrastructural sustainability of the retrieval process. The choice of architecture directly determines not only an answer’s scope but also the engine’s ecological footprint, shifting the cost of discovery onto the open web.

5 Results: Server-Web Interaction

To probe the underlying mechanics of real-time fetching, we conducted a server-side audit on a controlled website. The outcomes, summarized in Table 4, move beyond observing *what* pages are fetched to reveal *how* these engines interpret and interact with web content. The consistent behavior across models from the same provider suggests the use of shared, underlying retrieval tools.³

Table 4: Provider-level outcomes across tasks. ✓ : success; ✗ : failure; “partial”: task located but completion blocked.

Task	Outcome
Basic data (books)	Both: ✓
Multi-page (books)	Both: ✗ — only 5/6 books found.
Multi-page (directive)	ChatGPT: ✓ Claude: ✓ (but inconsistent)
Image (with metadata)	Both: ✓
Image (no metadata)	Both: ✗
Audio (quote)	Both: Partial ✓ (located only)
Traversal (recipe)	ChatGPT: Partial ✓ (located only) Claude: ✓
Large text (lecture)	Both: ✓
Caching	Both: ✓ observed (10–15 mins)

5.1 Textual Retrieval and Navigation

Our initial tests reveal that answer engines are effective at single-page data extraction but lack autonomous exploratory logic. In the *Basic Data Test*, both providers successfully retrieved structured information from a single target page. However, their limitations became apparent in the *Multi-Page Data Test*. When asked to aggregate information across the site, neither model could autonomously traverse links to find a sixth book located on a blog page.

This limitation was only overcome when the prompt was modified to explicitly state that a sixth book existed (*Multi-Page Data Test + Extra Info*). In this case, ChatGPT consistently found the missing entry, while Claude’s success was intermittent. This demonstrates a critical finding: LLM engines are not “browsers” in the exploratory sense. They operate as directed tools, capable of following a search result to a landing page and parsing it, but rarely deviating unless explicitly instructed by a carefully framed prompt. Their retrieval is a function of directive, not discovery.

5.2 Multimodal Processing [RQ2]

Our second set of experiments addresses [RQ2] (what types of data are retrieved and cited?) and finds that interaction with non-textual content is superficial, mediated entirely by textual cues.

In the *Media Data Test (with contextual metadata)*, models successfully “read” an image to report membership numbers. However,

³We evaluated both ChatGPT and Claude. Notably, the web search tools are consistent across models within each provider: ChatGPT models use `web.run()`, while Claude models use `browse()`. Thus, provider-level comparisons are the focus of this section.

they did so by parsing the associated text, not by analyzing the image itself. When metadata was absent, no images were downloaded or analyzed at all. Similarly, in the *Audio Test*, models could identify the audio file’s existence via its textual description but made no attempt to download or transcribe it.

These results show that the engines’ multimodal capabilities are largely an illusion based on clever metadata parsing. They do not “see” images or “hear” audio; they read the text surrounding them. This makes their ability to process non-textual information highly brittle and dependent on well-structured, descriptive HTML.

5.3 Data Volume and Caching [RQ4]

Finally, we examined data handling and persistence, revealing a “brute-force” approach to downloads and only minimal caching, which directly answers [RQ4]. In the *Data Size Test*, both providers willingly downloaded an entire 380 MB page to extract a single detail. However, their success was contingent on the detail’s position; when it was moved further down, the models failed, likely due to context window limitations. This highlights a core inefficiency: the engines download indiscriminately but can only process a fraction of the content.

The *Caching Test* revealed that both providers implement a rudimentary, short-term cache, persisting data from the same IP address for approximately 10–15 minutes. While this provides some session-level continuity, it is far from the sophisticated, cross-user caching of traditional search engines and does little to mitigate redundant data transfer at web scale.

5.4 Takeaway

Our server-side audit reveals that LLM “browsing” is not an intelligent, exploratory process. Instead, it is a highly constrained mechanical workflow that operates on three principles: (1) Follow the entry point from a search index and parse that page. (2) Rely on textual metadata to infer the content of non-textual media. (3) Brute-force download entire pages, even massive ones, but process them within strict context limits. This limited, non-persistent, and metadata-dependent behavior explains the significant inefficiencies and redundancies observed in our user-side study and reinforces the infrastructural burden of real-time fetching.

6 Quantifying the Per-Query Footprint

Our findings show that LLM engines touch many more pages than human searchers, but a raw URL count is insufficient to grasp the full infrastructural cost. To bridge this gap, we introduce a human-centric cost model that translates retrieval behavior into two concrete metrics: (1) the number of pages a human would likely open to achieve the same search breadth, and (2) the actual network bandwidth consumed. This allows us to quantify the practical implications of different retrieval architectures.

6.1 A Human-Equivalent Cost Model

Our model is anchored in established empirical studies of human search behavior. First, we establish a baseline for a single information-seeking task, using the finding from Venkit et al. [30] that users click an average of 3.80 sources per task. Second, we account for the declining engagement within a session. Humans rarely click as

gpt-5	11.9	1025.8 KB	Unknown	Unknown
gpt-5-thinking	8.8	789.5 KB	427.7 KB	8.9
gpt-o4-mini	9.1	657.6 KB	329.6 KB	4.6
opus-4.1	4.4	265.3 KB	169.8 KB	1.0
sonnet-4	4.7	326.6 KB	199.4 KB	1.0
	Cited URLs Count per query (avg)	Cited URLs Size per query (avg)	Forecasted URLs Size per query (avg)	Search Strings per query (avg)

Figure 8: Overall (aggregated across all categories) comparison of retrieval costs. Heatmap shows the average cited URL count, total size of cited URLs, forecasted retrieval size, and search-string count per query. A category-specific version of this heatmap for each model is included in the Appendix.

many links on their third query reformulation as they do on their first. Following Jiang et al. [15], we model this with a 40-60% decay rate in clicks over a typical 5-query session.

Calibrating a geometric decay model (see Appendix B) to these findings yields the expected number of human page opens, $H(k)$, for a task requiring k LLM search strings:

$$H(k) = 1.96 \cdot (1 + 0.5 + 0.5^2 + \dots + 0.5^{k-1}). \quad (1)$$

This formulation provides a transparent bridge between machine behavior (the number of internal searches, k) and a human-equivalent action (page opens). For network cost, we measure the compressed (e.g., gzip) payload of all cited URLs, reflecting true data transferred.

6.2 Architectural Choices Drive Network Costs

When viewed through this lens, the architectural divide between GPT and Claude translates into starkly different infrastructural costs, as shown in Figure 8.

Cited Data Volume. The most direct cost is bandwidth. Here, the differences are dramatic. GPT-5 is the most data-intensive, transferring over 1~MB of cited content per query on average. GPT-o4-mini and gpt-5-thinking occupy a middle ground at 650~790~KB. In sharp contrast, Claude models, constrained by their design to retrieve only 10 results (as seen in Figure 7), maintain a much smaller footprint of less than 330~KB per query.

Human-Equivalent Forecast. The cost of breadth is even clearer in our human-equivalent forecast. GPT-5-thinking’s iterative, multi-query strategy (averaging 8.9 search strings) results in a forecasted retrieval size of 430~KB, similar to the more balanced gpt-o4-mini (329.6 KB). Claude models, which rarely issue more than one search string, remain more conservative in both actual and forecasted costs, highlighting the efficiency gained by sacrificing exploratory breadth.

6.3 Takeaway

This analysis moves beyond abstract URL counts to reveal the tangible network cost of architectural decisions. The pursuit of expansive, multi-query synthesis, primarily seen in GPT models, comes at a steep and direct infrastructural cost, consuming orders of magnitude more bandwidth than Claude’s constrained, single-pass approach. This highlights the defining trade-off of the current real-time search paradigm: expansive coverage is achieved by offloading a disproportionate bandwidth and processing burden onto the open web, creating significant questions about the long-term sustainability of the model.

7 Discussion

Taken together, these findings reveal a fundamental and unsustainable trade-off. The promise of real-time, conversational synthesis is currently being fulfilled by offloading an enormous infrastructural burden onto the open web. This model imposes a hidden tax on content creators, especially smaller ones, who must bear the costs of excessive bandwidth consumption and server load.

Unlike traditional crawlers, which are designed for politeness and efficiency over weeks, real-time fetching is immediate, duplicative, and resource-intensive. Without fundamental changes, this paradigm risks corroding the very ecosystem it depends on, creating an operational and ethical challenge for the web community. If the future of search is AI-driven, it cannot be built on a foundation of unsustainable fetching.

Addressing this dilemma requires a concerted effort from both engine providers and the research community, necessitating a shift in priorities. First and foremost, **efficiency must be a first-class metric** alongside accuracy. Providers should aim to satisfy users with the minimum necessary retrieval, benchmarked against a more sustainable human-equivalent effort of 2 to 4 pages per query. This focus on efficiency would be supported by technical and policy innovations. Architecturally, **system-wide caching** must go beyond session/IP-level reuse to reduce redundancy at scale. In parallel, a commitment to **radical transparency and standardization** is crucial. The opaque retrieval policies hinder accountability, and providers should disclose practices to enable cooperative standards, including extensions to robots.txt (e.g., llms.txt).

Ultimately, the transformative potential of LLM search is undeniable, but its current trajectory raises serious questions of scalability and fairness. The path forward requires moving beyond answer quality alone and embracing a new paradigm where efficiency, transparency, and sustainability are recognized as essential conditions for the future of information access.

8 Conclusion

This paper presented an empirical measurement of how LLM-based web search engines interact with the open web. Using large-scale user-side instrumentation and controlled server-side auditing, we quantified their reliance on existing indices, bandwidth consumption, and caching behavior. Our results show that current LLM answer engines act as meta-search systems that heavily reuse conventional search results while repeatedly fetching pages in real time, leading to redundancy and significant infrastructural overhead.

By linking these findings to a human-equivalent cost model, we showed that architectural choices among LLM engines directly shape their sustainability footprint. GPT-style systems favor broad exploratory retrieval at the cost of efficiency, while Claude-style systems achieve efficiency through stricter retrieval limits—revealing trade-offs between coverage and network sustainability.

The key insight is that sustainability must become a first-class design goal alongside accuracy and transparency. Future work should explore scalable caching frameworks, cooperative retrieval standards (e.g., llms.txt), and benchmarks that treat efficiency as a measurable property. Viewing LLM answer engines as infrastructural participants in the web ecosystem can help guide AI-driven search toward models that are both intelligent and sustainable.

References

- [1] Daria Alexander, Wojciech Kusa, and Arjen P. de Vries. ORCAS-I: Queries annotated with intent using weak supervision. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.
- [2] Andrei Broder. A taxonomy of web search. *SIGIR forum*, 36(2):3–10, 2002.
- [3] Li Chen, Rohan Gupta, et al. Scrapers selectively respect robots.txt: Evidence from large-scale web measurements. *arXiv preprint arXiv:2505.21733*, 2025.
- [4] Zehui Chen, Kuikun Liu, Qiuchen Wang, Jiangning Liu, Wenwei Zhang, Kai Chen, and Feng Zhao. Mindsearch: Mimicking human minds elicits deep ai searcher, 2025.
- [5] Nick Craswell, Daniel Campos, Bhaskar Mitra, Emine Yilmaz, and Bodo Billerbeck. ORCAS: 18 million clicked query-document pairs for analyzing search. *arXiv preprint arXiv:2006.05324*, 2020.
- [6] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, Bodo Billerbeck, and Ellen M. Voorhees. Overview of the trec 2019 deep learning track. In *Text REtrieval Conference (TREC)*, 2019.
- [7] Yifan Ding, Matthew Facciani, Amrit Poudel, Ellen Joyce, Salvador Aguina, Balaji Veeramani, Sanmitra Bhattacharya, and Tim Weninger. Citations and trust in llm generated answers. *arXiv preprint arXiv:2501.01303*, 2025. v1.
- [8] Marcos Fernández-Pichel, Juan C. Pichel, and David E. Losada. Search engines, large language models or both? evaluating information seeking strategies for answering health questions, 2024.
- [9] Google Developers. Puppeteer documentation, 2025. Accessed: 2025-10-07.
- [10] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024.
- [11] Jeremy Howard. LLMs.txt: A standard for llm-friendly website content. Online proposal, available at <https://llmstxtmanager.com/llms-txt-standards-proposal-jeremy-howard-ceo-answer-ai/>, 2024.
- [12] Martin Huschens, Martin Briesch, Dominik Sobania, and Franz Rothlauf. Do you trust chatgpt? – perceived credibility of human and ai-generated content. *arXiv preprint arXiv:2309.02524*, 2023.
- [13] Imperva Inc. 2024 imperva bad bot report: Nearly half of internet traffic is bots, 2024. Press Release, Apr. 16, 2024.
- [14] Imperva Inc. 2025 imperva bad bot report: Bots surpass human traffic for the first time, 2025. Press Release, Apr. 15, 2025.
- [15] Jiepu Jiang, Daqing He, and James Allan. Searching, browsing, and clicking in a search session: changes in user behavior by task and over time. In *Proceedings of the 37th International ACM SIGIR Conference on Research & Development in Information Retrieval*, SIGIR '14, pages 607–616, New York, NY, USA, 2014. Association for Computing Machinery.
- [16] Carolin Kaiser, Jakob Kaiser, René Schallner, and Sabrina Schneider. A new era of online search? a large-scale study of user behavior and personal preferences during practical search tasks with generative AI versus traditional search engines. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, Yokohama, Japan, 2025. Association for Computing Machinery.
- [17] Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents, 2025.
- [18] Angeliki Lazaridou, Elena Gribovskaya, Wojciech Stokowiec, and Nikolai Grigorev. Internet-augmented language models through few-shot prompting for open-domain question answering, 2022.
- [19] Haiwen Li and Sinan Aral. Human trust in ai search: A large-scale experiment. *arXiv preprint arXiv:2504.06435*, 2025. v1.
- [20] Zhengxiao Liu, Xuming Ma, Zhiyuan Liu, and Maosong Sun. Webglm: Towards reliable web-enhanced question answering. *ACM Transactions on Information Systems (TOIS)*, 42(4), 2024.
- [21] Lucas Mearian. What's next for generative ai in 2025?, 2024. Computerworld (Online), Dec. 31, 2024.
- [22] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Karl Cobbe, Charlie Dao, David Farhi, William Fedus, Leo Gao, Christopher Hesse, Scott Khan, et al. Webgpt: Browser-assisted question-answering with human feedback, 2021.
- [23] Christopher Olston and Marc Najork. Web crawling. *Foundations and Trends in Information Retrieval*, 4(3):175–246, 2010.
- [24] Qwen Team. Qwen chat, 2025. AI assistant powered by the Qwen series models. Accessed: 2025-10-05.
- [25] Lee Rainie. Close encounters of the ai kind: The increasingly human-like way people are engaging with language models, 2025. Imagining the Digital Future Center (Elon University), Survey Report, Mar. 12, 2025.
- [26] Dejan Ravšelj et al. Higher education students' perceptions of chatgpt: A global study of early reactions. *PLOS ONE*, 2025.
- [27] Stefan Savage and colleagues. Awareness, agency, and blocking: User experiences with ai crawlers. In *Proceedings of the ACM Internet Measurement Conference (IMC)*. ACM, 2025.
- [28] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [29] Nikhil Sharma, Q. Vera Liao, and Ziang Xiao. Generative echo chamber? effect of llm-powered search systems on diverse information seeking. In *CHI '24: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, New York, NY, USA, 2024. Association for Computing Machinery. See arXiv:2402.05880 for the preprint version.
- [30] Pranav Venkit, Damian Blasi, Daniel Romero, and Robert West. Search engines in an ai era: The false promise of factual and verifiable source-cited responses, 2024.
- [31] Pranav Narayanan Venkit, Philippe Laban, Yilun Zhou, Yixin Mao, and Chien-Sheng Wu. Search engines in the AI era: A qualitative understanding to the false promise of factual and verifiable source-cited responses in LLM-based search. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT '25)*, Athens, Greece, 2025. ACM.
- [32] Ante Wang, Linfeng Song, Qi Liu, Haitao Mi, Longyue Wang, Zhaopeng Tu, Jin-song Su, and Dong Yu. Search-engine-augmented dialogue response generation with cheaply supervised query production. *Artificial Intelligence*, 319:103874, June 2023.
- [33] WHATWG. Server-sent events, 2025. Living Standard. Accessed: 2025-10-07.
- [34] Colin White et al. Livebench: A challenging, contamination-free LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [35] World Wide Web Consortium. Http archive (har) format, 2025. W3C Recommendation. Accessed: 2025-10-07.
- [36] Jiazhan Wu, Can Xu, Yiming Zhang, Yuxiang Zhang, Danyang Zhang, Weizhe Lv, Yichong Xu, Qing Lin, Xiaoyu Tang, Wayne Xin Zhao, Zhoujun Li, and Houfeng Wang. Freshllms: Refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13748–13765, 2024.
- [37] xAI. Grok, 2025. Grok chat assistant; links to web app at grok.com and apps on X, iOS, and Android. Accessed: 2025-10-05.
- [38] Kai-Cheng Yang. News source citing patterns in AI search systems, 2025.
- [39] Xiangrui Zeng, Yanzhe Deng, Ziyang Chen, Yuxuan Shen, Zhiqiang Li, Haoming Wang, Wenxin Zhang, Zhiyuan Liu, and Maosong Sun. Autowebglm: A large language model-based web navigating agent. *arXiv preprint arXiv:2404.03648*, 2024.
- [40] Wei Zhang, Arjun Kumar, et al. ai.txt: Towards web standards for regulating ai agent interactions. *arXiv preprint arXiv:2505.07834*, 2025.

A Server-Web Test Webpages

This appendix provides a detailed overview of the structure of the test website that was constructed to evaluate the retrieval mechanisms of our answer engines. The controlled website environment was designed to emulate realistic web content containing multiple data types, page structures, and interlinking behaviors. Each component was selected to test specific retrieval scenarios — ranging from link-following and multimodal interpretation to handling large data payloads and multi-page navigation. The following subsections describe each page and its intended testing role.

A.1 Overview

The test site was organized into a homepage, a content-rich Book Club page, an extensive Lecture Session page, and a sequence of interconnected blog posts. Each of these webpages played a specific role in benchmarking retrieval and reasoning behavior. The site provided a reliable and repeatable environment for observing how the system parsed, categorized, and prioritized content when responding to queries or extracting data under controlled conditions.

A.2 Homepage

Figure 9 shows the homepage of the test site. It served as the entry point and contained links to the three major content categories: *Book Club*, *Lecture Session*, and *Blogs*. The design was intentionally minimal, focusing on clear navigation pathways and simple hyper-link structures. This page was used primarily to verify the retrieval system’s ability to recognize and follow hyperlinks accurately, as well as to measure how it categorized and indexed different URL types. By monitoring the frequency of accessed versus presented links, we could assess whether the answer engine prioritized content according to relevance or followed links uniformly.



Figure 9: Homepage: entry point linking to the main content sections.

A.3 Book Club Page

Figure 10 presents the Book Club section, which was designed to include multiple data modalities within a single structured layout. The page combines plain textual descriptions, embedded images, and short audio playback elements. Each image includes captions or file-names to simulate partial contextual cues, while visual plots—such as bar charts and pie charts—represent the presence of quantitative data. This diversity in content format provided a useful means of testing whether the retrieval system could discern contextually

relevant elements when presented with mixed data. The Book Club page thus served as the central test case for multimodal reasoning, allowing us to observe how language models integrate textual and visual cues to extract information effectively.

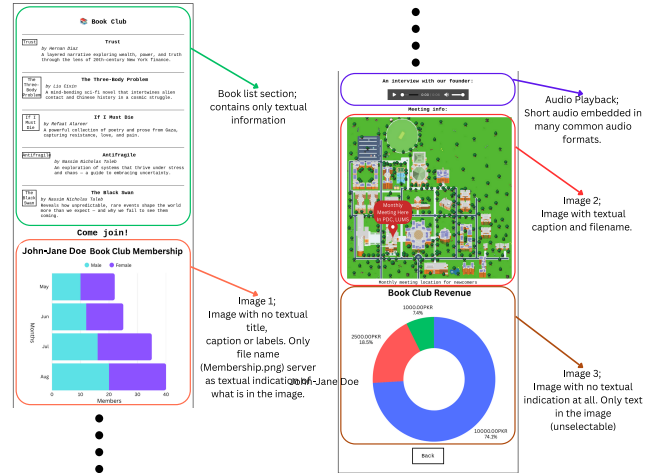


Figure 10: Book Club page: multiple content types used to test multimodal retrieval.

A.4 Lecture Session Page

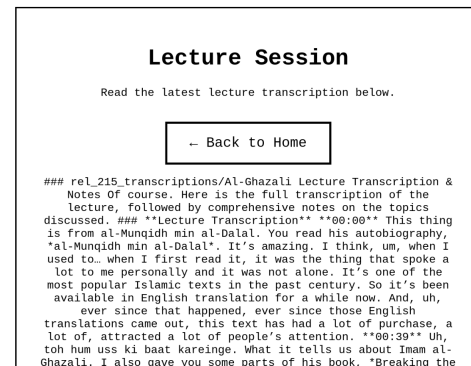


Figure 11: Lecture Session page: large-scale text document used to evaluate handling of extensive data.

The *Lecture Session* page, shown in Figure 11, contains a full transcription of an academic-style lecture. At approximately 380 MB, this is the largest page on the site and was designed to stress-test the retrieval engine’s capacity for handling large text-based documents. The goal was to evaluate whether the model would attempt to load or summarize such extensive data directly, or whether it would defer retrieval operations due to resource constraints. The page also served as a benchmark for assessing efficiency when processing deeply nested or extended text structures. Its use allowed us to monitor both retrieval latency and the precision of extracted content when only portions of a large file were required for answering a query.

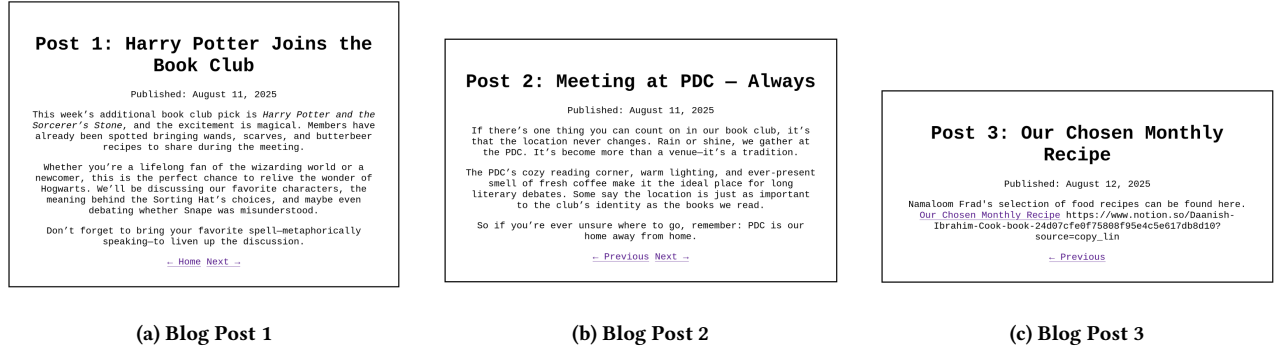


Figure 12: Blog pages used to test navigation and sequential retrieval across interconnected content.

A.5 Blog Pages

The blog section consisted of three sequentially linked posts that simulated a typical reading and navigation pattern across a multi-page structure. As illustrated in **Figure 12**, each post contained a publication date, a short article, and navigation buttons linking to the next or previous entry. These posts were designed to test whether the answer engine could traverse pages in the correct logical order and whether it would maintain contextual coherence when referencing information spread across multiple links. The experiment also tracked how frequently the model revisited prior pages, providing insight into whether its retrieval pattern resembled a structured crawl or an opportunistic traversal.

A.6 Summary

Taken together, the webpages illustrated in Figures 10–12 provide a comprehensive testing framework for retrieval benchmarking. The homepage evaluated basic link detection and categorization, the Book Club page tested multimodal and contextual retrieval, the Lecture Session page assessed performance with large documents, and the blog posts examined navigational consistency. Through this controlled environment, we could measure both retrieval accuracy and behavioral tendencies across data scales and page structures, offering a grounded understanding of how language-based retrieval systems engage with realistic, web-like contexts.

B Forecasting Methodology

This appendix provides further detail on how the forecasting formula in the main text was derived.

B.1 Notation and Data Sources

- k : number of searches (query reformulations) executed by the LLM for a single user prompt.
- $H(k)$: forecasted number of human-opened pages for that prompt.
- **Human anchor**: Venkit et al.[30] report a mean of **3.80 citations clicked per task** in the Google condition, which we interpret as pages opened. This serves as our calibration target.

- **Decay across queries**: Jiang et al.[15] show that the number of unique clicks per query drops by about **40–60%** as sessions progress, with a typical session involving around **5 queries**. We adopt a mid-point decay factor of $r = 0.5$ and set $k_0 = 5$.

B.2 Decay Model

We assume the number of clicks per query follows a geometric decay:

$$c_i = c_1 \cdot r^{i-1}, \quad (2)$$

where c_i is the expected human clicks on the i th query, c_1 is the clicks on the first query, and r is the decay factor. The cumulative human clicks across k queries is then:

$$H(k) = \sum_{i=1}^k c_1 \cdot r^{i-1}. \quad (3)$$

B.3 Calibration

We calibrate c_1 such that a 5-query task produces the human anchor of 3.80 clicks:

$$H(k_0) = \sum_{i=1}^5 c_1 \cdot 0.5^{i-1} = 3.80. \quad (4)$$

Solving yields $c_1 \approx 1.96$.

B.4 Final Formula

Substituting c_1 and $r = 0.6$ gives the simplified forecasting model:

$$H(k) = 1.65 \cdot (1 + 0.5 + 0.5^2 + \dots + 0.5^{k-1}). \quad (5)$$

B.5 Interpretation and Usage

This formulation means that humans are expected to open about 1.96 pages on the first query, with each subsequent query producing 50% as many clicks as the previous one. Over five queries, this totals 3.80 clicks, consistent with Venkit et al.[30]. For any prompt where the LLM executes k searches, $H(k)$ provides a transparent, literature-grounded estimate of the number of pages a human would have opened.